DESIGN AND ANALYSIS OF ALGORITHMS · UNIT 2

Quick Sort

Why do C++, Go and Java all lean on it?

Ammey Kuraaday
Roll No. B107

Shreyash Javale
Roll No. B34

1959: Hoare flips merge sort around

1959

Tony Hoare, a visiting student in Moscow, invents it while working on machine translation.



	Merge sort	Quick sort
Divide	Trivial: cut the array in the middle	**The real work: partition around a pivot**
Conquer	Sort both halves recursively	Sort both parts recursively
Combine	**The real work: merge two sorted halves**	Nothing. The array is already in order

Quick sort moves the effort from combine to divide.

Two pointers, one swap, one pivot home

```
1  Partition(A, low, high)
2    pivot = A[low]
3    i = low + 1
4    j = high
5    while true
6      while A[i] <= pivot: i++
7      while A[j] >  pivot: j--
8      if i < j: swap(A[i], A[j])
9      else: break
10   swap(A[low], A[j])
11   return j
```

i Scans right. Stops at the first element greater than the pivot.

j Scans left. Stops at the first element not greater than the pivot.

swap If i < j, swap A[i] and A[j], then keep scanning.

end Once they cross, swap the pivot into A[j]. That index is final.

Plot twist: a bounds bug in the notes. If the pivot is the largest value, i runs past the end: on [85 80 75 70] (indices 5 to 8) it reads A[9]. Fix: **`while i <= high and A[i] <= pivot`**

Follow the pivot 65 to its home

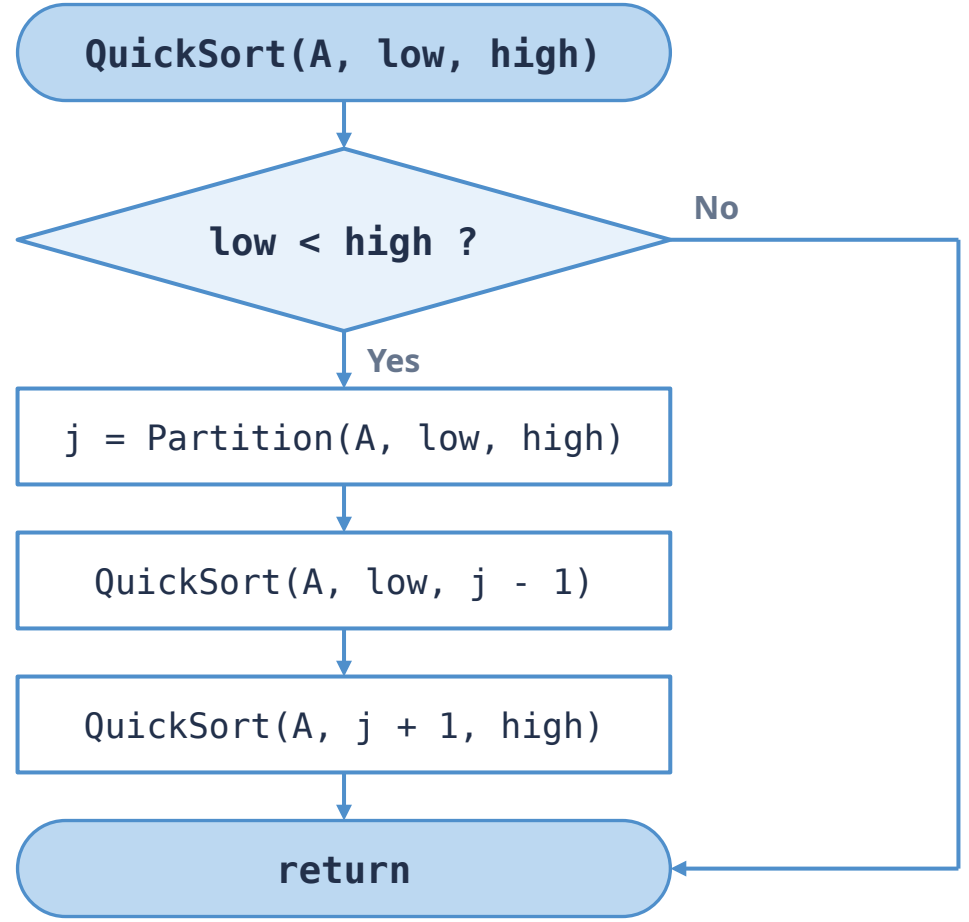
	0	1	2	3	4	5	6	7	8	
start	**65**	70	75	80	85	60	55	50	45	Pivot = A[0] = 65
swap 1	**65**	**45**	75	80	85	60	55	50	**70**	i stops at A[1] = 70, j stops at A[8] = 45: swap
swap 2	**65**	45	**50**	80	85	60	55	**75**	70	i stops at A[2] = 75, j stops at A[7] = 50: swap
swap 3	**65**	45	50	**55**	85	60	**80**	75	70	i stops at A[3] = 80, j stops at A[6] = 55: swap
swap 4	**65**	45	50	55	**60**	**85**	80	75	70	i stops at A[4] = 85, j stops at A[5] = 60: swap
finish	60	45	50	55	**65**	85	80	75	70	**i = 5 has passed j = 4: swap the pivot with A[4]**

pivot swapped pair final row: left four ≤ 65, right four ≥ 65

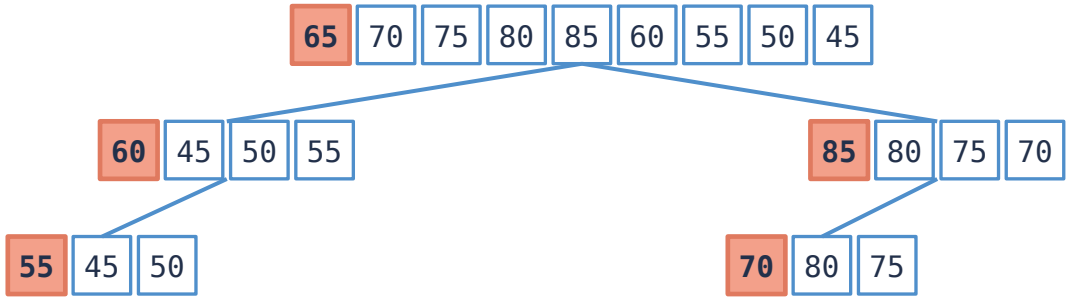
Do it again on both sides, until nothing is left

```
1  QuickSort(A, low, high)
2    if low < high
3      j = Partition(A, low, high)
4      QuickSort(A, low, j - 1)
5      QuickSort(A, j + 1, high)
```

No combine step. When both calls return, the whole range is already sorted.



OUR EXAMPLE, CALL BY CALL (coral = pivot)



Everything depends on the pivot

BEST CASE

$T(n) = 2T(n/2) + n$

$\Theta(n \log n)$

The pivot lands in the middle every time: log n levels, n work on each.

AVERAGE CASE

random pivot or random input

$\approx 1.39\, n \log n$

About 39% above the minimum any comparison sort needs.

WORST CASE

$T(n) = T(n-1) + n$

$\Theta(n^2)$

Pivot always smallest or largest, e.g. sorted input: 12,497,500 comparisons for 5,000 values.

Pivot choice	Comparisons	Weakness
First element (our notes)	≈ 1.39 n log n on random data	**Sorted input gives n^2**
Random element	≈ 1.39 n log n expected, any input	Needs random numbers
Median of three	≈ 1.19 n log n	Tricky inputs still exist

Why the world still sorts this way

Introsort

`C++ std::sort`

Switches to heap sort if recursion gets too deep, and to insertion sort on tiny ranges.

pdqsort

`Go sort, Rust sort_unstable`

Spots bad pivots and ordered patterns, then changes strategy, so the worst case is rare.

Dual-pivot

`Java Arrays.sort`

Two pivots give three parts. Used for primitive arrays since Java 7.

Algorithm	Best	Average	Worst	Extra space	Stable	In place
Quick sort	n log n	n log n	**n^2**	log n	**No**	Yes
Merge sort	n log n	n log n	n log n	n	Yes	No
Heap sort	n log n	n log n	n log n	1	No	Yes

Why it wins

In place, cache friendly, small constant factor, worst case engineered away. The price: not stable, so use merge sort when equal keys must keep their order.

Thank you

Questions?

Partition does the work. The pivot lands home. Then repeat.

Ammey Kuraaday · B107

Shreyash Javale · B34